SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT)
Constituent of Symbiosis International (Deemed University), Pune
(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of india)
Re-Accredited by NAAC with 'A++' Grade
॥वसुधैव कुटुम्बकम्॥

Name : Atharva Ambilwade
PRN : 25070122055
Class : CSE
Batch : 2025-29

Assignment : 28

Subject: Programming with Java

ASSIGNMENT No : 28

TITLE : Develop a Java application demonstrating Remote Method Invocation (RMI) using stub and skeleton components for remote communication.

Problem Statement : Develop a Java application demonstrating Remote Method Invocation (RMI) using stub and skeleton components for remote communication.

Learning Objectives : To implement Remote Method Invocation for distributed Java applications.

Theory :

Remote Method Invocation (RMI) is a Java technology that enables an object running on one Java Virtual Machine (JVM) to invoke methods of an object running on another JVM located on a different machine. RMI simplifies distributed application development by allowing remote objects to behave like local objects. It consists of remote interfaces, remote object implementations, the RMI registry, and client-server communication. The registry maintains references to remote objects, allowing clients to locate and invoke them. RMI provides platform-independent communication, object serialization, and automatic network handling, making it suitable for distributed enterprise applications.

Code :

```java
import java.rmi.*;
import java.rmi.registry.*;
import java.rmi.server.*;

interface Hello extends Remote {
    String message() throws RemoteException;
}

class HelloImpl extends UnicastRemoteObject implements Hello {

    HelloImpl() throws RemoteException {
        super();
    }

    public String message() throws RemoteException {
        return "Hello from RMI Server";
    }
}

public class A28 {
    Run main | Debug main | Run | Debug
    public static void main(String[] args) {
        try {
            LocateRegistry.createRegistry(1099);

            HelloImpl obj = new HelloImpl();

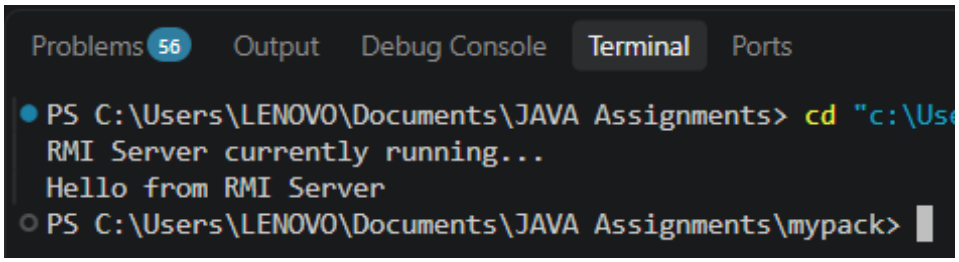
            Naming.rebind("HelloService", obj);

            System.out.println("RMI Server currently running...");
            System.out.println(obj.message());

            System.exit(0);
        }
        catch (Exception e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Output :

Output :



Conclusion : This program demonstrates Java Remote Method Invocation (RMI), which allows a client to invoke methods on a remote object. The Hello interface defines the remote method, while HelloImpl implements it. The RMI registry is created on port 1099, and the Naming.rebind() method registers the remote object under the name HelloService, enabling clients to access its functionality remotely.

Exercise

1. **Create a remote calculator service using Java RMI**

Ans) Calculator.java

Client-Server > Calculator.java > Java > Calculator

```java
import java.rmi.*;

public interface Calculator extends Remote {
    int add(int a, int b) throws RemoteException;
    int subtract(int a, int b) throws RemoteException;
}
```

Server

Client-Server > CalculatorImplServer.java > ...

```java
import java.rmi.*;
import java.rmi.registry.*;
import java.rmi.server.*;

class CalculatorImpl extends UnicastRemoteObject implements Calculator {
    CalculatorImpl() throws RemoteException {
        super();
    }

    public int add(int a, int b) throws RemoteException {
        return a + b;
    }

    public int subtract(int a, int b) throws RemoteException {
        return a - b;
    }
}

public class CalculatorImplServer {
    Run main | Debug main | Run | Debug
    public static void main(String[] args) {
        try {
            LocateRegistry.createRegistry(1099);
            Calculator obj = new CalculatorImpl();
            Naming.rebind("CalculatorService", obj);
            System.out.println("Calculator Server Started");
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

Client

```java
import java.rmi.*;

public class CalculatorClient {
    Run main | Debug main | Run | Debug
    public static void main(String[] args) {
        try {
            Calculator c = (Calculator)
                Naming.lookup("rmi://localhost/CalculatorService");

            System.out.println("Addition: " + c.add(10, 5));
            System.out.println("Subtraction: " + c.subtract(10, 5));
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

Output :

```
Problems 64  Output  Debug Console  Terminal  Ports                                   java - Client-Server

PS C:\Users\LENOVO\Documents\JAVA Assignments> cd .\Client-Server\
PS C:\Users\LENOVO\Documents\JAVA Assignments\Client-Server> javac Calculator.java Calculatorl
mplServer.java CalculatorClient.java

PS C:\Users\LENOVO\Documents\JAVA Assignments\Client-Server> java CalculatorImplServer
Calculator Server Started
```

```
PS C:\Users\LENOVO\Documents\JAVA Assignments\Client-Server> java CalculatorClient
Addition: 15
Subtraction: 5
PS C:\Users\LENOVO\Documents\JAVA Assignments\Client-Server>
```

2. **Create a Java application using RMI where the server converts temperature values between Celsius and Fahrenheit. The client sends input temperature and receives converted output.**

Ans) Server

Client-Server > TempServer.java > Java > TempServer

```java
import java.rmi.*;
import java.rmi.registry.*;
import java.rmi.server.*;

interface Temp extends Remote {
    double convert(double t, int choice) throws RemoteException;
}

class TempImpl extends UnicastRemoteObject implements Temp {
    TempImpl() throws RemoteException {
        super();
    }

    public double convert(double t, int choice) throws RemoteException {
        if (choice == 1)
            return (t * 9 / 5) + 32;
        else
            return (t - 32) * 5 / 9;
    }
}

public class TempServer {
    Run main | Debug main | Run | Debug
    public static void main(String[] args) {
        try {
            LocateRegistry.createRegistry(1099);
            Naming.rebind("TempService", new TempImpl());
            System.out.println("Temperature Server Started");
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

Client :

```
Client-Server > TempClient.java > Java > TempClient
1   import java.rmi.*;
2   import java.util.Scanner;
3
4   public class TempClient {
        Run main | Debug main | Run | Debug
5       public static void main(String[] args) {
6           try {
7               Temp t = (Temp) Naming.lookup("rmi://localhost/TempService");
8               Scanner sc = new Scanner(System.in);
9
10              System.out.println("1. Celsius to Fahrenheit");
11              System.out.println("2. Fahrenheit to Celsius");
12              System.out.print("Enter choice: ");
13              int choice = sc.nextInt();
14
15              System.out.print("Enter temperature: ");
16              double temp = sc.nextDouble();
17
18              System.out.println("Converted temperature: " + t.convert(temp, choice));
19              sc.close();
20          } catch (Exception e) {
21              System.out.println(e);
22          }
23      }
24  }
```

Output ;

```
Problems 68   Output   Debug Console   Terminal   Ports                                   java - Client-Server

PS C:\Users\LENOVO\Documents\JAVA Assignments> cd Client-Server
PS C:\Users\LENOVO\Documents\JAVA Assignments\Client-Server> javac TempServer.java TempClient.java
PS C:\Users\LENOVO\Documents\JAVA Assignments\Client-Server> java TempServer
Temperature Server Started

PS C:\Users\LENOVO\Documents\JAVA Assignments\Client-Server> java TempClient
Converted temperature: 122.0
PS C:\Users\LENOVO\Documents\JAVA Assignments\Client-Server> java TempClient
1. Celsius to Fahrenheit
2. Fahrenheit to Celsius
Enter choice: 2
Enter temperature: 90
Converted temperature: 32.22222222222222
PS C:\Users\LENOVO\Documents\JAVA Assignments\Client-Server>
```